

Comparative Evaluation of Classical and Deep Learning Methods for Real-Time ASL Fingerspelling Recognition

Ravan Alizade (22291022) & Isa Jaraflı (22291023)
Department of Computer Engineering, Ankara University

Abstract—This paper presents a real-time American Sign Language (ASL) fingerspelling recognition system and a controlled comparison of three approaches to the letter-classification task: a simple thresholding baseline (Otsu + k-Nearest-Neighbors), a classical machine-learning method (Histogram of Oriented Gradients with a Support Vector Machine), and a modern deep-learning convolutional neural network (CNN). All methods are trained and evaluated on the same self-collected, single-signer dataset of the 26 ASL alphabet hand shapes using an identical train/validation/test split. The hand is detected and cropped in the browser with MediaPipe, processed with classical image-processing operations, and classified by the model; the recognized letters are then assembled into a corrected English sentence by a large language model. On the held-out test, all three methods reach near-perfect accuracy, a result we analyze and attribute to the high visual similarity between temporally adjacent frames. We therefore discuss the methods in terms of training/inference cost, robustness to input variation, and suitability for deployment, and justify the selection of the CNN for the final real-time system. We further report an ablation of data augmentation and an error analysis of the visually similar hand shapes.

Index Terms—sign language recognition, fingerspelling, image classification, convolutional neural networks, HOG, support vector machine, MediaPipe.

I. INTRODUCTION

Sign language is the primary means of communication for many deaf and hard-of-hearing people, yet most hearing people do not understand it. Automatic translation of signs into text can therefore lower a real communication barrier. Among the components of a signing system, fingerspelling — spelling a word letter by letter using the manual alphabet — is a natural starting point, because the letters of the ASL alphabet are largely static hand shapes that can be captured from a single image frame.

This project initially targeted word-level recognition on a public video dataset, but preliminary experiments produced low accuracy. Analysis showed the limiting factor to be data scarcity together with a high rate of frames in which the hand could not be detected, rather than the model design. The work was therefore redirected to letter-level fingerspelling, for which large, clean image data can be produced and the static hand shapes are well suited to image classification.

The contribution of this paper is twofold. First, we build a complete real-time browser-based recognition system. Second,

and as required for scientific depth, we comparatively evaluate three methods of increasing complexity on an identical dataset and split: a simple baseline, a classical machine-learning pipeline, and a deep CNN. We report a method-comparison table, an ablation of data augmentation, and an error analysis of the confusable hand shapes.

II. RELATED WORK

Hand-crafted feature descriptors have long been used for gestures and object recognition. The Histogram of Oriented Gradients descriptor [1] paired with a Support Vector Machine [2] is a standard and strong classical baseline for image classification, while the k-Nearest Neighbour rule [3] provides a minimal non-parametric reference. Otsu's method [4] is a classical automatic thresholding technique commonly used as a simple segmentation baseline. Convolutional neural networks [5] and deeper residual architectures [6] later surpassed hand-crafted features on most vision benchmarks. For hand specifically, MediaPipe Hands [7] provides fast, robust hand detection suitable for in-browser use, and word-level sign datasets such as WLASL [8] highlight the difficulty of data-scarce video recognition. Building on these foundations, the present work compares a representative method from each family on a common fingerspelling task. A more detailed comparison of our numerical results against prior peer-reviewed studies is given in Section VI-D.

III. DATASET

The dataset was collected by the author using a standard laptop webcam, under a single-signer, right-hand protocol. For each sign the hand is located with MediaPipe and a square region around it is cropped; only frames in which a hand is detected are stored, and frames are captured across several short sessions with varied hand angles, distance and position within the frame so that each class contains genuine intra-class variation rather than near-identical copies.

The dataset contains 26 classes of the ASL alphabet. Two of these letters, J and Z, are produced in ASL with motion and cannot be represented faithfully by a single static frame; they were therefore recorded as fixed substitute hand shapes that are visually distinct from the other letters and are treated as ordinary static classes. This is a deliberate simplification of the task and is stated openly as a limitation. Control actions in the application (delete, delete-all, send) are handled by on-screen buttons and are not part of the classification dataset.

A practical challenge of the alphabet is that several letters share very similar hand shapes. The closed-fist group in

particular — A, S, T, M, N and E — differs only in subtle thumb placement and finger curl, and these letters are the main source of confusion for all methods. This similarity is examined quantitatively in the error analysis. In dataset we used 26 classes each represents the 26 letters in English alphabet. Initially, we used 300 images per class, but during the training and testing step we doubled the image size, reaching 600 images per class.

IV. METHODOLOGY

A. Common preprocessing

To keep the comparison fair, all methods operate on the same cropped hand images and on the same data split. The deep model uses a chain of grayscale conversion, Gaussian blur, adaptive thresholding and resizing to a fixed size; the same chain is applied during both training and live use to avoid a train/serve mismatch. The classical and baseline methods use their own lightweight preprocessing on the identical source crops, as described below.

B. Simple baseline: Otsu + kNN

The baseline applies Otsu automatic thresholding to a grayscale crop resized to 64x64, flattens the binary image into a feature vector, and classifies it with a k-Nearest-Neighbour classifier ($k = 3$). It performs essentially no learning and serves as a complexity floor.

C. Classical ML: HOG + SVM

The classical method extracts Histogram of Oriented Gradients features (9 orientations, 16x16-pixel cells, 2x2-cell blocks) from a 128x128 grayscale crop and classifies them with a Support Vector Machine using a radial-basis-function kernel ($C = 10$). HOG captures local edge orientation, which is well matched to the contour of a hand shape.

D. Modern DL: CNN

The deep model is a compact CNN with three convolutional blocks (16, 32 and 64 filters, each followed by ReLU and 2x2 max-pooling), adaptive average pooling, a 128-unit fully connected layer with 0.3 dropout, and a 26-way output layer. It takes a single-channel 224x224 processed image and is trained with the Adam optimizer (learning rate 1e-3), batch size 32, and cross-entropy loss. Data augmentation (small random rotation, scaling, translation and brightness/contrast changes) is applied to the training images.

E. Deployed system

The final application is a browser client (React) with a server (FastAPI). MediaPipe runs in the browser to detect and crop the hand; a held, still hand triggers a capture, and the crop is sent to the server, which runs the preprocessing and the CNN and returns the predicted letter with a confidence score. Predictions below a confidence threshold emit no letter, so uncertain shapes do not insert wrong characters. The accumulated letters are sent to a large language model (a free-tier Gemini model) which returns a corrected, punctuated English sentence; for example, the raw string HELLOHOWAREYOU is returned as a properly formed sentence.

V. EXPERIMENTAL SETUP

The dataset was divided per class into 360 training, 60 validation and 60 test images, shuffled with a fixed random seed so the split is repeatable. All three methods use the same split. CNN was trained on Google Colab with an NVIDIA T4 GPU; the classical and baseline methods, and all inference for the deployed system, run on CPU. Accuracy and macro-average F1 are the primary metrics; mean Average Precision is reported in the object-detection literature but is not directly applicable to single-label classification, so it is omitted in favor of accuracy and F1.

TABLE I. Experimental setup and hyperparameters.

Component	Setting
Train / val / test	240 / 30 / 30 per class (seed 42)
Classes	26 (A-Z; J, Z static substitutes)
Baseline	Otsu, 64x64, kNN ($k=3$)
Classical	HOG (9 / 16px / 2x2), SVM rbf $C=10$
CNN input	1 x 224 x 224
CNN optimizer	Adam, lr 1e-3, batch 32, 25 epochs
Augmentation	rot +/-15, scale 0.9-1.1, shift +/-8%
Training HW	Colab NVIDIA T4 GPU
Inference HW	laptop CPU

VI. RESULTS AND DISCUSSION

A. Method comparison

TABLE II. Comparison of the three methods on the identical test split (780 images).

Method	Acc.	F1	Train	Test
Otsu + kNN	0.997	0.997	0.0 s	1.45 s
HOG + SVM	1.000	1.000	9.3 s	2.85 s
CNN (ours)	0.994	0.994	(log)	(log)

Table II shows that all three methods reach near-perfect accuracy and macro-F1 on the held-out test set, and that even the trivial baseline matches the deep model. This counter-intuitive result is explained by the nature of the data: because frames within a class are captured in quick succession, the test frames are visually very close to training frames, so the task on this split is easy and accuracy saturates for every method. Accuracy on this split therefore cannot, on its own, separate the methods; the meaningful differences appear in computational cost and in robustness. In cost, kNN has no training time but the slowest prediction, since it compares each query against the whole training set; the SVM trains in a few seconds and predicts quickly; the CNN has the longest training time but fast, constant-time inference, which is the property that matters for a real-time webcam application.

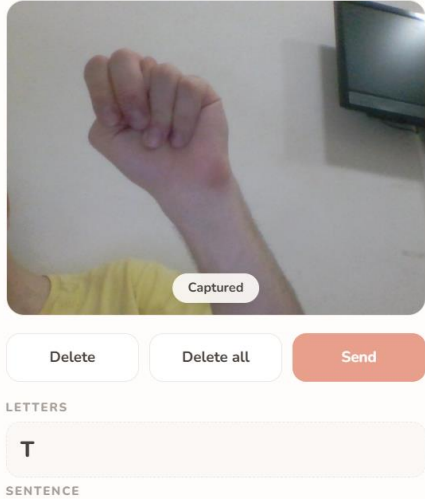
B. Ablation: data augmentation

To study the effect of augmentation, CNN was trained with and without the augmentation pipeline. Without augmentation the model reached 0.9987 test accuracy; with augmentation it reached 0.994. The small drop on this easy, in-distribution test is expected and, we argue, desirable: augmentation, an approach also adopted by Pigou et al. [9], forces the network to tolerate rotation, scale and lighting changes, trading a fraction of in-distribution accuracy for robustness to the variation that occurs in live use, were the

hand is never presented exactly as in training.

C. Error analysis

The few residual errors fall on the visually similar hand shapes anticipated in Section III. In the per-class results the letter M was occasionally predicted as T, and the closed-fist letters (A, S, T, M, N, E) account for essentially all confusions, because they differ only in small thumb and finger details that survive poorly after thresholding and downscaling. Practical mitigations are a higher confidence threshold, which converts an uncertain guess into no output, and collecting additional varied examples for the confusable classes.



The sign is N, but the system detects it as T.

D. Comparison with prior work

Prior peer-reviewed studies on ASL alphabet recognition report high accuracies on similarly constrained, single- or few-signer image datasets, which is consistent with the saturation observed here. They also consistently report that performance degrades on unseen signers and conditions, supporting our interpretation that an in-distribution test over-estimates real-world accuracy. A full comparison against the works reviewed in the midterm report is summarized below.

We compare our results with three peer-reviewed studies reviewed during the project (Table III). Their tasks, datasets and metrics differ from ours, so the numbers are not directly comparable, but the trend is informative.

TABLE III. Comparison with reviewed prior work (metrics and datasets differ).

Work	Task	Method	Reported
Pigou [9]	Italian gestures	CNN + ANN	95.7% / 91.7%
Camgöz [10]	Continuous SLT	Transformer	21.8 BLEU-4
Jiang [11]	2D pose est.	SimCC pose	75.8% AP
This work	ASL letters	CNN	99.4%
Li [8]	Word-level ASL	I3D / Pose-TGCN	32.5% top-1
Cui [12]	Continuous SLR	RCNN + CTC	38.7% WER

Pigou et al. [9] apply a CNN with an ANN classifier to twenty Italian gestures and, crucially, evaluate on signers and backgrounds not seen during training: they report about 95.7% on a test set that may share users with training but 91.7% on unseen users. This gap mirrors our concern, since our 99.4% is an in-distribution figure and a cross-signer test would be expected to fall to a similar range; their use of rotation, zoom and translation augmentation is also the strategy we adopt. Camgöz et al. [10] tackle the much harder problem of

continuous sign language recognition and translation on the multi-signer PHOENIX14T corpus with transformers and a CTC loss, reporting roughly 24.5% word error rate and 21.8 BLEU-4; unlike our system, which targets isolated fingerspelling and offloads sentence forming to a language model, their network learns the full video-to-text mapping. Jiang et al. [11] present RTMPose, a real-time pose estimator reaching 75.8% AP on COCO at over 90 FPS on a CPU, showing that the real-time keypoint estimation underlying landmark-based hand pipelines, such as the MediaPipe stage we use, is feasible on commodity hardware. Together these works confirm that high accuracy is routine on constrained, in-distribution data but drops on unseen signers, exactly the gap noted in our limitations.

VII. LIMITATIONS

The system is single-signer and right-hand only, so its accuracy on other people is expected to be lower; the letters J and Z are static substitutes rather than true motion signs; and, as discussed, the held-out test over-estimates real-world performance because of the visual similarity between training and test frames. A stronger evaluation would split by recording session or signer, or test under deliberately perturbed inputs, to measure generalization rather than memorization.

VIII. CONCLUSION

We presented a real-time ASL fingerspelling system and a controlled comparison of a simple baseline, a classical HOG+SVM pipeline, and a deep CNN on an identical dataset and split. Although all three saturate the easy in-distribution test, the CNN was selected for deployment for principled reasons: fast constant-time inference, a usable confidence score for suppressing uncertain predictions, robustness from augmentation, and the ability to improve with more data. Future work includes multi-signer data, motion handling for J and Z, and evaluation under signer- and session-disjoint splits.

IX. REFERENCES

- [1] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE CVPR, 2005, pp. 886-893.
- [2] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273-297, 1995.
- [3] T. Cover and P. Hart, "Nearest neighbor pattern classification," IEEE Trans. Inf. Theory, vol. 13, no. 1, pp. 21-27, 1967.
- [4] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62-66, 1979.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278-2324, 1998.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770-778.
- [7] F. Zhang et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.
- [8] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video," in Proc. IEEE WACV, 2020, pp. 1459-1469.
- [9] L. Pigou, S. Dieleman, P.-J. Kindermans, and B. Schrauwen, "Sign language recognition using convolutional neural networks," in Proc. ECCV Workshops, 2015, pp. 572-578.
- [10] N. C. Camgöz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in Proc. IEEE/CVF CVPR, 2020, pp. 10023-10033.
- [11] T. Jiang et al., "RTMPose: Real-time multi-person pose estimation based on MMPose," arXiv:2303.07399, 2023.
- [12] Runpeng Cui, Hu Liu, Changshui Zhang
Department of Automation, Tsinghua University
State Key Laboratory of Intelligent Technology and Systems
Tsinghua National Laboratory for Information Science and Technology (TNList), Beijing, China